SIS3204Y(5): Natural Language Processing

Topic: Feature Extraction & Text Encoding (Based on "Week 3 - Feature Extraction.pptx.pdf")
Level: MSc AI / Level 2 | **Format:** Revision Questions & Official Answer Key

Question 1: Foundational Concepts [3 Marks]

Define **Feature Extraction** in the context of NLP and explain briefly why raw text cannot be fed directly into machine learning models.

> **OFFICIAL ANSWER KEY**
>
> - **Definition (1 Mark):** Feature extraction is the process of converting raw, unstructured text into a set of meaningful numerical features (vectors) that a machine learning model can understand and use.
> - **Why raw text fails (2 Marks - 1 mark per valid point):**
> - Models perform mathematical operations (addition, dot products), which are not defined on text strings.
> - Words have no natural numerical meaning or inherent magnitude (e.g., "cat" is not numerically closer to "dog" than to "car").

Question 2: Pipeline Architecture [6 Marks]

Map out the complete **Text Representation Pipeline** by listing the seven sequential steps required to transform raw text into numerical features ready for machine learning models.

> **OFFICIAL ANSWER KEY**
>
> - **Step 1 (0.5 Marks):** Raw Text (Original text collection)
> - **Step 2 (1 Mark):** Text Preprocessing (Cleaning text, removing noise, lowercasing)
> - **Step 3 (1 Mark):** Tokenization (Splitting cleaned text into individual words/tokens)
> - **Step 4 (1 Mark):** Vocabulary Building (Creating a list of unique tokens)
> - **Step 5 (1 Mark):** Indexing / Mapping (Assigning an integer index to each token)
> - **Step 6 (1 Mark):** Encoding (Converting indices into a numerical format, e.g., one-hot)
> - **Step 7 (0.5 Marks):** Numerical Feature Vectors (Final vectors ready for ML models)

Question 3: Encoding Limitations [4 Marks]

While **Integer (Label) Encoding** is simple and memory-efficient, it introduces severe issues when used directly with linear or distance-based neural models. Describe TWO major problems with integer encoding.

> **OFFICIAL ANSWER KEY**
>
> - **Problem 1: Implies False Order (2 Marks):** The model incorrectly assumes a numeric scale (e.g., if apple=0 and zebra=4, the model assumes zebra > apple). This artificial order does not exist in language.
> - **Problem 2: Implies False Distance (2 Marks):** The model thinks the numeric distance between IDs has semantic meaning (e.g., assuming $|4 - 0| = 4$ means zebra is semantically far from apple). These distances mean nothing semantically and mislead algorithms like k-NN or linear regression.

Question 4: Enumeration [5 Marks]

State FIVE significant limitations of using **One-Hot Encoding** for large-scale natural language processing tasks.

> **OFFICIAL ANSWER KEY**

Question 5: Applied Process [5 Marks]

Given the sentence: *"the dog chased the cat"*, perform One-Hot Encoding. Assume the vocabulary is sorted alphabetically. Construct the vocabulary table with indices, and draw the final resulting One-Hot Matrix representing the entire sentence.

OFFICIAL ANSWER KEY

Part A: Vocabulary & Indexing (2 Marks)

- `cat` (Index 1)
- `chased` (Index 2)
- `dog` (Index 3)
- `the` (Index 4)

(Vocabulary Size $V = 4$; Sentence Length $N = 5$)

Part B: Final One-Hot Matrix (3 Marks)

Word / Target	the	dog	chased	the	cat
cat (1)	0	0	0	0	1
chased (2)	0	0	1	0	0
dog (3)	0	1	0	0	0
the (4)	1	0	0	1	0

Note: Matrix dimensions are $V \times N$ (4 × 5). Award full marks if rows and columns correctly map binary presence.

Question 6: Explanatory Concept [4 Marks]

Explain the **Distributional Hypothesis** and state how it serves as the foundational principle for learning Word2Vec embeddings.

OFFICIAL ANSWER KEY

- **The Hypothesis (2 Marks):** The Distributional Hypothesis states that words that occur in similar contexts tend to have similar meanings.
- **Application to Word2Vec (2 Marks):** Word2Vec leverages this by analyzing the context in which words appear across a large corpus. By predicting context words from a target word (or vice versa), the model automatically learns co-occurrence patterns, placing words with similar contexts closer together in a dense vector space.

Question 7: Comparative Analysis [6 Marks]

Using a side-by-side comparison, contrast **Sparse Representations** (e.g., One-Hot, Bag-of-Words) with **Dense Representations** (e.g., Word2Vec) across three distinct technical parameters.

OFFICIAL ANSWER KEY

Award 2 marks for each correctly contrasted parameter from the table below:

Parameter	Sparse Representations (Traditional)	Dense Representations (Modern)
Vector Size & Content	Very high-dimensional (size of vocabulary). Composed mostly of zeros (sparse).	Low-dimensional (typically 50-500). Composed of real-valued numbers without zeros.
Capturing Meaning & Similarity	No notion of meaning. All words are treated as equally distant and orthogonal.	Captures semantic meaning. Words with similar meanings are placed close together in vector space.
Efficiency & Scalability	High memory usage and slower computations due to massive waste processing zeros.	Low memory usage and faster computations. Generalizes well to unseen contexts.

Question 8: Synthesis and Limitations [8 Marks]

Despite being a foundational NLP technique, Word2Vec has critical limitations. Discuss these limitations specifically focusing on **Polysemy**, **Out-of-Vocabulary (OOV) words**, and **Contextual Staticness**. How do modern models (like BERT) address these issues?

OFFICIAL ANSWER KEY

- **Polysemy (2 Marks):** Word2Vec fails to handle words with multiple meanings. A word like "bank" (river vs. finance) gets only one single collapsed vector, mixing different senses together.
- **Out-of-Vocabulary (OOV) Words (2 Marks):** If a word was not seen during the training phase, Word2Vec cannot generate a vector for it. It lacks a fallback mechanism for entirely new words.
- **Static Embeddings (2 Marks):** Word2Vec generates a static representation; a word is given the exact same vector regardless of the sentence context it appears in (e.g., "apple" the fruit vs. "apple" the company).
- **Modern Solutions (2 Marks):** Modern models like BERT provide *contextual (dynamic) embeddings*, where the vector changes based on the surrounding sentence, solving polysemy and staticness. They also use subword tokenization (e.g., WordPiece) to assemble representations for OOV words from known parts.

Question 9: Architectural Comparison [6 Marks]

Word2Vec utilizes two complementary architectures to learn embeddings: **CBOW (Continuous Bag of Words)** and **Skip-Gram**. Compare these two architectures in terms of their Prediction Direction, Training Speed, and Ideal Use Cases.

OFFICIAL ANSWER KEY

Award 2 marks for each correctly mapped row:

Aspect	CBOW (Continuous Bag of Words)	Skip-Gram
Prediction Direction	Predicts a single **Target word** from its surrounding **Context words** (Context → Target).	Predicts multiple **Context words** from a single **Target word** (Target → Context).
Training Speed & Data	Faster to train. Highly data efficient; works very well with smaller to medium datasets.	Slower to train. Requires significantly larger datasets to perform optimally.
Ideal Use Cases	Best for frequent words and capturing broad, general semantic meaning.	Best for capturing specific meanings and modeling rare or infrequent words.